Contents

Software Design Document — music-room **1**
1. Introduction . . . 1
2. References . . . 2
3. Stakeholders and design concerns . . . 2
4. Context viewpoint . . . 3
5. Composition viewpoint . . . 3
6. Logical viewpoint — domain model . . . 5
7. Dependency viewpoint . . . 6
8. Information viewpoint . . . 6
9. Interface viewpoint . . . 8
10. Interaction viewpoint . . . 11
11. State dynamics . . . 13
12. Concurrency viewpoint . . . 13
13. Security design . . . 14
14. Deployment and resource viewpoint . . . 22
15. Design rationale . . . 25
16. Requirements traceability . . . 27
17. Open design issues . . . 32
Related . . . 44

Software Design Document — music-room

Consolidated design view of the system, organised by the viewpoints of IEEE 1016.

Status: partly implemented. S0 through S2, S5, S6 and S7 are merged, and S3 is still open: identity, credential rate limits, the audit log, bearer authentication, own-profile editing, the Playlist Editor, Track Vote and Control Delegation with Playback all exist. The three services subject V.2 asks for are all built. Still absent: the catalog (S4), the mail adapter (S1a), the vault (S1b), the OpenAPI UI (S3a), OpenID Connect (S8) and the capacity evidence (S9). **§16 is the authority for what is built** — it says which rows that covers and which are still design only; `docs/progress/PROGRESS.json` carries the counted figures. Every section below describes what is decided, and marks what is not with `UNKNOWN - requires decision`. A guess is never substituted for a decision.

---

1. Introduction

1.1 Purpose

One document an evaluator, a new teammate, or a future maintainer can read to understand how the system is designed and why. It consolidates the design; it does not replace the layers that own each part.

1.2 Scope

The `music-room` backend and the contract it exposes to clients. The React Native client is out of the current work scope — only the constraints it places on the backend appear here ([[ADR-0002-mobile-react-native]]).

1.3 Audience

Peer evaluators, team members, and any agent working in this repository.

1.4 Authority order

When two sources disagree:

`42 subject -> ADR -> standard -> this document -> note -> code comment`

This document **consolidates**; it is not the source of truth for anything. Rationale lives in ADRs, enforceable rules live in `docs/technical-guide/`, requirements live in the subject.

1.5 Relationship to the other layers

Layer	Owns	This document does
`brain/constraints/`	what is required	quote and trace it
`brain/decisions/`	why a choice was made	reference it, never restate the rationale
`brain/architecture/`	the working design notes	consolidate them into one readable view
`docs/technical-guide/`	enforceable rules	reference, never copy
`docs/owasp/`	the security conformance target and its evidence	reference, never copy
`MusicRoom/`	what the code does	describe intent only

---

2. References

Ref	Source
R1	42 subject "Music Room", v6 — `docs/subject/en.subject tr.pdf`, distilled in [[subject-requirements]]
R2	[[decisions-index]] — ADR-0001 through ADR-0006
R3	[[technical-guide-index]] — 19 engineering standards
R4	IEEE 1016 — software design description, used here as the section structure
R5	RFC 9110 (HTTP semantics), RFC 9457 (Problem Details), RFC 9700 (OAuth 2.0 BCP), OpenID Connect Core 1.0
R6	OWASP ASVS 5.0.0, vendored at `docs/owasp/upstream/asvs-5.0.0/`; conformance scope in [[asvs-conformance]]
R7	NIST SP 800-63B — password and verifier requirements

---

3. Stakeholders and design concerns

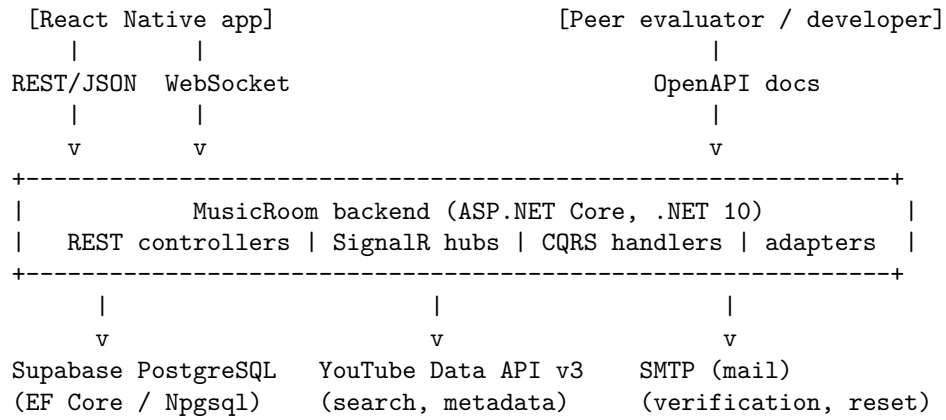
Stakeholder	Primary concern	Addressed in
End user at an event	the queue reflects the group's votes, immediately	§11, §12
Event owner	control over who may vote and edit	§9.2, §13.2
Peer evaluator	every mandatory requirement is met and defensible	§16

Stakeholder	Primary concern	Addressed in
Team developer	knowing where code goes and which rule applies	§5, §7
Operator	the system can be run, measured, and recovered	§14

---

4. Context viewpoint

4.1 System context



4.2 External actors and interfaces

External element	Direction	Protocol	Owned by
React Native client	inbound	HTTPS REST/JSON, WebSocket	us (deferred)
Supabase PostgreSQL	outbound	PostgreSQL wire protocol	vendor, managed
YouTube Data API v3	outbound	HTTPS REST	vendor
YouTube player SDK	client-side only	—	vendor
Google / Facebook OIDC	outbound	OAuth 2.0 + PKCE	vendor
Mail provider	outbound	HTTPS API	hosted free tier – [[ADR-0013-mail-provider]]

Design constraint. No client ever reaches Supabase or YouTube Data API directly. Subject R1 V.3 makes the backend the single guardian of truth, and V.6 makes it the single authorisation gate. A direct client connection would route around both. See [[ADR-0006-datastore]].

---

5. Composition viewpoint

5.1 Physical composition

One repository, two application roots – [[ADR-0020-monorepo-layout]]:

Root	Contents	Owner
`MusicRoom/`	backend, .NET 10	backend developer
`MusicRoomApp/`	React Native client	UI developer

One clone is the whole submission, which subject VII requires. Neither root writes into the other; the contract between them is [[endpoint-inventory]] and the generated `docs/api/openapi.json`. `reference/` is read-only study material, never built, referenced or copied from – see [[reference-narchitecture]].

Five projects, per [[clean-architecture-cqrs]]:

Project	Responsibility
`Domain`	entities, value objects, enums, domain rules, domain exceptions
`Application`	commands, queries, handlers, validators, pipeline behaviours, abstractions
`Persistence`	`DbContext`, entity configurations, repositories, EF Core migrations
`Infrastructure`	YouTube adapter, mail, token and OTP services
`WebAPI`	controllers, SignalR hubs, middleware, DI composition, OpenAPI

Layout mirrors the reference repository – [[ADR-0011-solution-layout]]:

```
MusicRoom/
  MusicRoom.sln
  src/
    corePackages/   Core.Application | Core.CrossCuttingConcerns | Core.Persistence | Core.Security
    MusicRoom/      Domain | Application | Persistence | Infrastructure | WebAPI
  tests/
    <Project>.Tests/
```

The shape is copied; no file is. A type enters `corePackages` only if it is domain-agnostic.

5.2 Logical composition — bounded contexts

Context	Owns	Subject ref
Identity	user, credentials, email verification, password reset, social links	V.1
Profile	profile fields with per-field visibility, music preferences	V.1
Event (Track Vote)	event, attendee, track suggestion, vote, licence policy	V.2.1
Playlist (Editor)	playlist, collaborator, ordered tracks	V.2.3
Delegation (Control)	device, control grant	V.2.2
Playback	playback session, now-playing pointer, queue advance	V.2.1, V.2.2
Catalog	YouTube track lookup and cache	V.2, III
Audit	per-action record: platform, device, app version	V.6

Subject R1 V.2 requires at least 2 of 3 services. **All three ship** – [[ADR-0010-service-scope]]. Identity, Profile, Catalog and Audit are required regardless.

Consequence: the location+time licence mode is mandatory, decided in [[ADR-0014-location-time-licence]]. If the schedule slips, Control Delegation is dropped first – it is the smallest, and removing it still leaves two services, satisfying V.2.

---

6. Logical viewpoint — domain model

Detail source: [[domain-model]]. Field-level detail is deliberately absent until the first migration exists.

6.1 Aggregates

Aggregate	Root	Key invariant
User	`User`	email unique; an unverified email cannot use services
Profile	`Profile`	every field carries one of `Public / Friends / Private`
Event	`Event`	one vote per user per track; ranking derives from vote count
Playlist	`Playlist`	position ordering stays total and gap-tolerant
Device	`Device`	a control grant belongs to exactly one device
PlaybackSession	`PlaybackSession`	at most one active session per device; `controllerUserId` is what a control grant changes
TrackRef	value object	a YouTube reference only — never media content

6.2 Shared enumerations

Enum	Values	Source
`Visibility`	`Public`, `Private`	R1 V.2.1, V.2.3
`LicenceMode`	`Everyone`, `InvitedOnly`, `LocationAndTimeRestricted`	R1 V.2.1, V.2.3
`PresenceProof`	`Geofence`, `EventCode`, `Both`	policy carried by `LocationAndTimeRestricted` – [[ADR-0014-location-time-licence]]
`ProfileFieldVisibility`	`Public`, `Friends`, `Private`	R1 V.1 — **implemented**, stored as `smallint` per field ([[ADR-0034-profile-storage-and-visibility]])
`FriendRequestStatus`	`Pending`, `Accepted`, `Declined`	R1 V.1 — **implemented**. There is no `Cancelled`: a withdrawn request is deleted
`MusicPreferenceKind`	`Genre`, `Artist`	R1 V.1 — **implemented**
`Platform`	`iOS`, `Android`, `Web`, `Other`	R1 V.6

The reader's relationship to a profile — self, friend or stranger — is **derived per request** from the token and the friends graph, never stored, so it is not an enumeration this table owns.

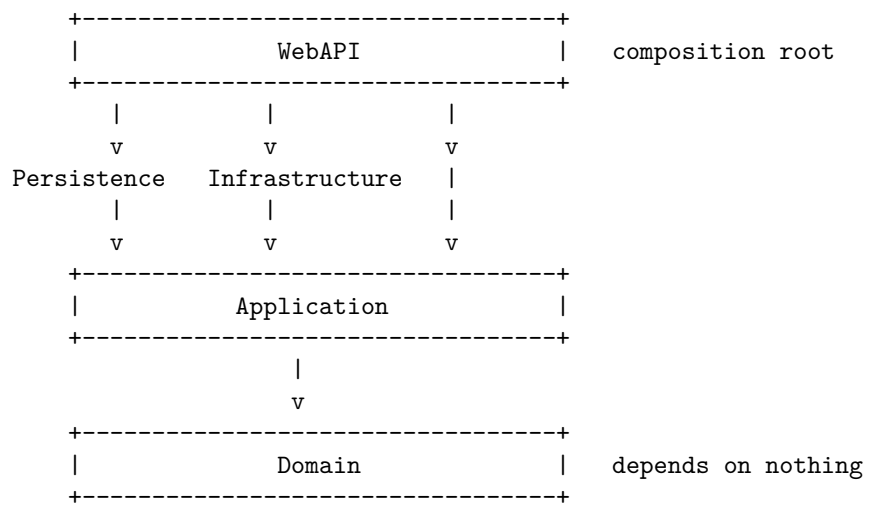
6.3 Deliberate exclusions

Audio and video content is never stored, cached, proxied or re-hosted. Only `TrackRef` metadata is persisted — [[ADR-0003-music-provider-youtube]].

---

7. Dependency viewpoint

7.1 The dependency rule



Enforced constraints, from [[clean-architecture-cqrs]]:

- `Domain` references no other project and no infrastructure package.
- `Application` references neither EF Core, nor ASP.NET Core, nor any vendor SDK.
- `WebAPI` never touches `DbContext` directly.
- A reference pointing outward is a build-breaking defect, not a style issue.

7.2 External dependency boundaries

External system	Isolated behind	Located in
YouTube Data API	one adapter, interface declared in `Application`	`Infrastructure`
SMTP	mail service abstraction	`Infrastructure`
Google / Facebook	provider token validation service	`Infrastructure`
PostgreSQL	repositories + `DbContext`	`Persistence`

---

8. Information viewpoint

8.1 Storage

Supabase-managed PostgreSQL, accessed through EF Core with the Npgsql provider ([[ADR-0006-datastore]]). Only the Postgres surface is used — Supabase Auth, Realtime, Storage and PostgREST are deliberately not adopted, because identity, realtime and the API are ours.

8.2 Constraints that carry invariants

Conventions and full rules: [[database-design-migrations]].

Constraint	Protects	Why it is in the database
`ux_votes_event_id_track_id_user_id`	one vote per user per track	a read-then-write check in C# is a race condition
`ux_users_email`	account uniqueness	concurrent registration
`xmin` concurrency token on `playlists`	reorder conflicts	last-writer-wins must be detectable
`xmin` concurrency token on `devices`	control takeover	two simultaneous claims must not both succeed

8.3 Common columns

Every table: `id`, `created_at_utc`, `updated_at_utc`. User-owned aggregates use soft delete (`deleted_at_utc`) so the audit trail survives. All instants are `timestamp with time zone`, written as UTC.

8.4 Ordering strategy

Playlist position uses sparse ordering keys with gaps, so inserting between two tracks does not rewrite the list. See §12.2.

8.5 Schema ownership and evolution

- EF Core migrations are the only mechanism that changes the schema. Editing the schema in the Supabase dashboard desynchronises migration history and is treated as an incident.
- Two connection strings: pooled (Supavisor) for the API, **direct** for migrations.
- Destructive changes follow expand -> migrate -> contract.
- Every migration is recorded in [[migrations-index]].
- **The schema tool does not start the application.** `MusicRoomDesignTimeDbContextFactory` builds the context for `dotnet ef` from `DATABASE_URL` alone, so `make migrate` and `make migration` depend on the one setting they use and on nothing else (SPEC-029). Before it, `dotnet ef` fell back to the web host and ran every start-up guard the host has: a `.env` with `MAIL_PROVIDER=brevo` and an empty `MAIL_FROM` stopped a migration. The guards stay as they are — they close threat T-26 — and the factory takes the schema off their path. A missing `DATABASE_URL` is still a refusal naming the key, because ADR-0026 gives it no default on either path.

`playlists` holds one row per playlist with `owner_id`, `visibility`, `licence` and `deleted_at_utc`, and uses PostgreSQL's `xmin` as its concurrency token — no version column exists, which is what makes the `ETag` of §5 free (SPEC-014). Deletion is soft, applied by a global query filter. `playlist_collaborators` holds one row per person named on a playlist with `can_edit`, unique per playlist and account. `playlist_tracks` holds one row per track with a `bigint position`, a sparse key with a nominal gap of 1000 so an insert between two tracks writes one row; the index on `(playlist_id, position)` is **not unique**, because a renumber rewrites a whole playlist in one batch and nothing needs the uniqueness — every track write first updates the playlist row, which is also what gives the track **list** the `ETag` a collection cannot carry itself (SPEC-015).

`playback_sessions` holds one row per session with its device, a `(source_type, source_id)` pair naming an event or a playlist, the now-playing id, and `ended_at_utc`. `ux_playback_sessions_device_id` is **filtered** on `ended_at_utc IS NULL`, which is how ADR-0017's "at most one active session per device" becomes a constraint the database keeps; history accumulates behind it. There is **no controller column** — who may command a session is read from its device — and **no foreign key on `source_id`**, because one column pair for two possible parents is the honest shape (SPEC-021).

`devices` holds one row per registered device with its owner, the three client values it reported, and `controller_user_id` — the control grant itself, nullable because null means the owner controls it. The

row's `xmin` is the concurrency token that makes two simultaneous claims resolve to one `200` and one `412`. `ux_devices_owner_id_platform_model` excludes the app version on purpose: a phone that updates its app is the same phone (SPEC-020).

`event_suggestions` holds one row per track suggested for an event, unique per event and video. `event_votes` holds one row per account per suggestion, unique on `(event_id, suggestion_id, user_id)` — **and there is no count column anywhere**: the queue counts these rows on every read, so no two writers can disagree about a ranking and no increment can be lost (SPEC-018). `event_codes` holds a SHA-256 digest and an expiry, not a plaintext code, and `event_code_redemptions` is unique per code and account, which is what makes "single use per account" a fact the database holds. A voter's submitted coordinates are compared against the event's centre and discarded — they reach no row and no log ([[ADR-0014-location-time-licence]]).

`events` holds one row per track-vote event with `owner_id`, `visibility`, `licence`, a UTC window, and four nullable policy columns carrying [[ADR-0014-location-time-licence]]'s configuration — `presence_proof` and a `numeric(9,6)` centre with a radius in metres. It uses `xmin` as its concurrency token and `deleted_at_utc` for soft delete, the same mechanisms the playlist aggregate uses. The coordinates are the **venue's**; a voter's are checked and discarded and never stored. `event_invitations` is the `member` gate on a private event, unique per event and account; `event_attendances` is a roster and is read by nothing that decides whether somebody may act (SPEC-017).

`friend_requests` holds one row per ordered pair, unique on `(sender_id, recipient_id)`, with a `smallint` status — pending, accepted or declined. `friendships` holds **one row per friendship** with the pair in canonical order, `user_a_id < user_b_id`, unique on the pair: symmetry is a property of the data rather than of two rows that could disagree, and the unique index is what makes the friendship two crossing accepts would create impossible (SPEC-012).

`profiles` holds one row per account with a value column and a `smallint` visibility column per tiered field — `real_name`, `bio`, `location`, `birth_date` — created with the account so no read path has to answer for a missing profile. `music_preferences` holds one row per named genre or artist, unique per account, kind and value. `display_name` stays on `users`: it is always public (SPEC-011).

Concrete table and column definitions live in the migrations and their `IEntityTypeConfiguration` files, which are authoritative — [[migrations-index]] is the list, and this section does not repeat it. For a table no migration has created yet it is still `UNKNOWN - requires decision`.

---

9. Interface viewpoint

9.1 REST API

Rules: [[rest-api-design]]. Documentation: [[openapi-documentation]].

Aspect	Design
Base path	`/api/v1`
Format	JSON, `camelCase`, UTF-8
Errors	RFC 9457 `application/problem+json`, one shape, with `traceId`
Collections	envelope `{ items, page, pageSize, total }`, page size capped
Timestamps	UTC ISO-8601
Privacy rule	a resource hidden from the caller returns `404`, never `403`
Auth	bearer JWT

Aspect	Design
Documentation	the generated document at `/openapi/v1.json` in **every** environment, plus an interactive page at `/swagger/index.html` on a **Development** host only — [[ADR-0035-swagger-ui-for-the-openapi-document]], SPEC-032. `AddOpenApi()` stays the only generator: the page is a reader of the same document CI drift-checks against `docs/api/openapi.json`, and `SwaggerGen` is deliberately absent so nothing can describe the API twice
Mandatory headers	`X-Client-Platform`, `X-Client-Device`, `X-Client-App-Version` on mutations
Documentation	OpenAPI 3.1 generated from code, exported to `docs/api/openapi.json`. The document is served at `/openapi/v1.json` in every environment
Documentation UI	**Swagger UI, Development only** – `Swashbuckle.AspNetCore.SwaggerUI` renders the document above; the generator stays `AddOpenApi()` so one producer describes the API. Slice S3a – [[ADR-0035-swagger-ui-for-the-openapi-document]]

The endpoint inventory is [[endpoint-inventory]] – around 60 endpoints across Identity, Profile, Catalog, Track Vote, Playlist Editor, Control Delegation, audit and operations, plus the client-to-server hub methods. Concrete request and response schemas come from the generated OpenAPI document, not from that file.

9.2 Realtime API

Rules: [[signalr-realtime]]. Rationale: [[ADR-0004-realtime-signalr]].

Hub	Group key	Server-to-client events
`EventHub`	`event:{eventId}`	`TrackSuggested`, `VoteChanged`, `QueueReordered`, `NowPlayingChanged`
`PlaylistHub`	`playlist:{playlistId}`	`TrackAdded`, `TrackRemoved`, `TrackMoved`, `CollaboratorChanged`
`ControlHub`	`user:{userId}`	`ControlGranted`, `ControlRevoked`, `ControlTakenOver`

Contract rules:

All three hubs are **implemented**: `PlaylistHub` at `/hubs/playlists` (SPEC-016), `EventHub` at `/hubs/events` (SPEC-019) and `ControlHub` at `/hubs/control` (SPEC-022), behind one broadcast queue, one pipeline behaviour and one audit filter. `ControlHub` alone has no invocable method — its group is a person, joined from the validated token on connect, so there is nothing for a client to ask for and nothing to audit (ADR-0019 audits hub methods).

- Hubs are transport only. Every hub method dispatches the **same** CQRS handler the REST endpoint uses, so the two transports cannot diverge.

- Every payload carries the aggregate id. A playlist payload also carries a monotonic `version`, which is the parent row's `xmin` and the token its `ETag` carries. An **event** payload carries the authoritative value of what changed instead – a written, scoped deviation recorded in [[ADR-0039-queue-broadcast-payloads]]: an event's `xmin` is already the `If-Match` of its metadata `PATCH`, and a `queue_version` column would reintroduce the shared mutable count §12.2 forbids. Either way a client converges, which is what the rule is for.
- Group membership is authorised server-side on every join and re-checked on reconnect.
- A broadcast is emitted only after the transaction commits. The mechanism is structural rather than remembered: a handler queues what happened, and a pipeline behaviour registered **outside** the one that owns the transaction drains that queue once it has committed – [[ADR-0038-realtime-broadcast-posture]].
- Client context is captured at the handshake from the query string, and a handshake without it is answered `400` by the same middleware that refuses a REST mutation without the headers – [[ADR-0019-hub-action-auditing]].

9.3 External interfaces

Interface	Usage	Design note
YouTube Data API v3	`videos.list` preferred; `search.list` only on real user input	**built, SPEC-025.** Quota is billed in units and `search.list` costs about a hundred times a `videos.list`, so one adapter reaches the provider and two cache levels stand in front of it: `IMemoryCache`, then `track_refs`, then YouTube — [[ADR-0023-catalog-cache]]. A searched query's id list is cached as well, in `track_search_cache`, so a repeated search re-issues nothing. Every call logs its unit cost, `403 quotaExceeded` and a provider `5xx` both answer `503`, and the pipeline carries a per-attempt timeout, a bounded retry on `5xx` and `429` only, and a circuit breaker. With no key configured the caches still answer
YouTube player SDK	client-side playback	backend decides what and when, never streams media
Google / Facebook	authorization code flow with PKCE	provider token validated server-side against JWKS; a client-supplied profile is never trusted
Mail provider	verification and reset mail	hosted free tier behind `IMailService`; API key server-side – [[ADR-0013-mail-provider]]

The catalog degrades rather than failing. `youtube-integration` requires an event to keep running when the provider does not: a stale `track_refs` row is served in preference to an error, and a write path — suggesting a track, adding one to a playlist — never calls the provider at all. It validates the video id's shape and stores it, and metadata is resolved on read. So a spent quota costs titles, not votes.

10. Interaction viewpoint

Representative flows. Steps are design intent, not implemented code.

10.1 Registration with email and password

```
1. Client POST /api/v1/auth/registrations
2. Validation behaviour rejects malformed input, including the 12-64 character
   and 72-byte UTF-8 password limits that BCrypt requires
3. Handler hashes the password with BCrypt at cost >= 12, persists an unverified User
4. Infrastructure sends a verification mail with a single-use, hashed, 24h token
5. Response 202, always -- identical whether or not the address is already registered.
   A 409 here would be an account-enumeration oracle; the mailbox tells the real story.
   The account cannot use services until verified
6. Client POST /api/v1/auth/email-verifications with the token
7. Handler marks the account verified and consumes the token
```

Algorithm: [[ADR-0007-password-hashing-bcrypt]]. Paths and status codes: [[endpoint-inventory]] §1.

Step 4 sends real mail since SPEC-024. `MAIL_PROVIDER` chooses the sender: `brevo` is the hosted provider of [[ADR-0013-mail-provider]], and `logging` is the Development-only sender that writes the link to the application log. Neither is a default — a host with no valid value does not start, which is what replaced the "Development only" guard and what closes issue O17 and threat T-26. The message carries the token and nothing else personal, as HTML and plain text, composed in this repository rather than in a vendor template.

The provider path is **verified end to end**, once, by hand on 2026-10-04: a registration answered `202`, Brevo accepted the message with `201`, and it arrived in a real mailbox. No automated test spends quota (SPEC-024 decision 7), so that run is the only evidence the call itself works, and it is recorded in SPEC-024's verification table rather than claimed by a test. `MAIL_API_KEY` is a **v3 API key**, not an SMTP key: the adapter calls `POST /v3/smtp/email`, which answers `401` to an SMTP key.

The sender is **Brevo's sandbox sender**, not a verified domain: ADR-0013 permits that for the defence provided it is stated rather than presented as production, and this is the statement. `VERIFICATION_LINK_BASE_URL` and `PASSWORD_RESET_LINK_BASE_URL` decide where the links point; with neither set the message carries the bare token, because a link to a machine nobody intended is worse than no link. One send, bounded at 10 seconds, with no retry: the resend endpoint is the retry path and it is rate-limited, and a failed send never rolls the registration back — that is what keeps the `202` above from becoming an enumeration oracle.

10.2 Social login

OpenID Connect, not bare OAuth 2.0 – [[ADR-0008-oidc-social-login]].

```
1. Device runs the OIDC authorization code flow with PKCE
2. Client POST /api/v1/auth/social with the provider ID TOKEN (never an access token)
3. Server validates, in order:
     a. signature against the provider JWKS from its discovery document
     b. iss matches the expected issuer
     c. aud matches our client id      <- stops token substitution
     d. exp and iat within tolerance
     e. nonce matches the one this server generated
     f. email_verified is true before any email-based account match
4. Server maps (provider, providerUserId) to a local user, or creates one
5. Server issues ITS OWN access + refresh token pair
```

```
   - a provider token is never accepted as an API credential
```

Provider asymmetry: Google is a full OIDC provider; Facebook is not uniformly OIDC (Limited Login issues an ID token, classic Facebook Login does not). The integration branches. See [[social-oauth]].

10.3 Casting a vote — the contended write path

```
1. Client POST /api/v1/events/{eventId}/tracks/{trackId}/votes
2. Authorisation behaviour evaluates ownership, then Visibility and LicenceMode
3. Rate limiting behaviour applies per account and per IP
4. Transaction behaviour opens the transaction
5. Handler inserts the vote row
   - the unique index is the arbiter; a duplicate insert fails here, not in C#
   - a duplicate is mapped to 409 Conflict, never silently ignored
6. Vote count is recomputed from the source rows, never incremented in memory
7. Transaction commits
8. ONLY THEN: EventHub broadcasts VoteChanged and QueueReordered to event:{eventId}
```

Step 8 after step 7 is the rule that prevents clients from seeing state the server rolls back.

10.4 Moving a track in a playlist

```
1. Client PATCH .../playlists/{id}/tracks/{trackId} with the target position,
   carrying the playlist's current concurrency token
2. Authorisation behaviour evaluates Visibility and LicenceMode for edit rights
3. Handler computes a new sparse ordering key between the neighbours
4. Save fails with DbUpdateConcurrencyException if another edit landed first
   - the handler reloads and reapplies, or returns 409
5. On commit, PlaylistHub broadcasts TrackMoved with the new version
6. Clients treat the event as a reconcile hint, tolerating duplicates and reordering
```

10.5 Delegating music control

```
1. Owner POST .../devices/{deviceId}/control-grants with the target friend
2. Handler transitions the Device control state under its concurrency token
3. A competing claim loses on the concurrency token - it must not silently win
4. ControlHub notifies user:{ownerId} and user:{granteeId}
```

10.6 Advancing the queue

```
1. Controller device detects the current track ended
2. POST /api/v1/playback-sessions/{id}/advances  { expectedNowPlayingId }
3. Authorisation behaviour verifies the caller is the session's controllerUserId
4. Handler picks the next item:
     Event    -> highest vote count, ties broken by earliest suggestion
     Playlist -> next ordering key
5. Write under the concurrency token; a stale expectedNowPlayingId returns the current
   state instead of skipping a second track
6. Commit
7. ONLY THEN: NowPlayingChanged broadcast to the source's group
```

The server is the arbiter of **what** plays; the client is the authority on **when** it ended. The backend cannot observe audio, and a `durationSeconds` timer drifts on pause and buffering. See [[ADR-0017-playback-session]].

10.7 Every mutating request — the mandatory audit path

```
1. Middleware reads X-Client-Platform, X-Client-Device, X-Client-App-Version
```

```
2. Missing headers on a mutating request -> 400 (a silent default would void R1 V.6)
3. Values are attached to the request scope
4. An audit behaviour writes one audit record per handled command
```

---

11. State dynamics

11.1 Account state

```
Registered(unverified) --verify--> Active
Active --reset password--> Active (all refresh tokens revoked)
```

An unverified account cannot use the three services.

11.2 Playback session state

```
None --create--> Playing(nowPlaying)
Playing --advance--> Playing(next)      server picks next; idempotent on expectedNowPlayingId
Playing --advance--> Ended              queue exhausted
Playing --delete--> None
```

If the controller device goes silent mid-track, nothing advances. Accepted: control can be taken over via `PUT /devices/{id}/control` and the owner can always revoke. No server-side watchdog – it would reintroduce the drift this design avoids.

11.3 Device control state

Single-row state machine on `Device`, guarded by a concurrency token:

```
Uncontrolled --claim--> ControlledBy(owner)
ControlledBy(owner) --delegate--> ControlledBy(grantee)
ControlledBy(x) --revoke--> ControlledBy(owner)
ControlledBy(x) --release--> Uncontrolled
```

Concurrent transitions are resolved by the token. Exact state names are settled when the Delegation slice is built.

---

12. Concurrency viewpoint

Subject R1 V.2.1 and V.2.3 both call out race conditions explicitly. They are graded, so they are designed here rather than discovered in review. Full notes: [[realtime-architecture]].

12.1 Hotspots

#	Hotspot	Failure if unhandled
1	two users vote on the same track simultaneously	duplicate votes, wrong ranking
2	two users reorder the same playlist simultaneously	divergent order, lost move
3	two devices claim control at once	two controllers, undefined playback
4	two clients advance the same playback session at once	a track is silently skipped

12.2 Mechanisms

Hotspot	Mechanism
1	unique index `(EventId, TrackId, UserId)`; duplicate detected by the database, mapped to `409`
1	vote counts recomputed inside the transaction or aggregated database-side — never read-modify-write
2	sparse ordering keys plus an `xmin` concurrency token, **taken from the playlist row and updated inside the same transaction as the track write**, so the guarded `UPDATE` is both the `If-Match` check and the lock that serialises reorders; the losing writer gets `412` and reloads (SPEC-015)
3	single-row state machine with a concurrency token; the losing writer must lose visibly
4	`expectedNowPlayingId` in the request plus the session's concurrency token; a stale expectation returns current state rather than advancing again. **Implemented and proven** (SPEC-021): twenty parallel advances with one token move the session once, and an exhausted queue ends it once

12.3 Governing principles

1. **The database is the arbiter, not the hub and not the application.**
2. **Broadcast after commit.** Never inside the transaction.
3. **Every realtime event is a reconcile hint.** Clients must tolerate duplicates and reordering.
4. **A sequential test proves nothing about a race.** Concurrency cases run in parallel and assert final database state — [[testing]].

---

13. Security design

Full rules: [[authentication-authorization]], [[application-security]], [[secrets-management]]. Design notes: [[auth-and-identity]]. Conformance target: [[asvs-conformance]] — ASVS 5.0.0 Level 1 for every applicable chapter, Level 2 for V6 Authentication, V7 Session Management, V8 Authorization, V9 Self-contained Tokens, V10 OAuth and OIDC, and V16 Security Logging and Error Handling.

13.1 Authentication

Element	Design
Password hashing	**BCrypt** (`BCrypt.Net-Next`), work factor >= 12, rehash on login when the stored cost is lower – [[ADR-0007-password-hashing-bcrypt]]

Element	Design
Password policy	12 to 64 characters, **<= 72 bytes UTF-8** (BCrypt truncates beyond that), no composition rules, no forced rotation (NIST SP 800-63B). Stated **once**, in `PasswordRules` — SPEC-034 — and applied by the register, reset and change validators to the property each command exposes, so the `422` body stays keyed `Password` on register and `NewPassword` on the other two. Three validators each declared it separately before
Password screening	Two local screens, no outbound call — [[ADR-0042-breached-and-context-password-screening]], SPEC-035. **Breached:** a bundled set of **46,146** entries, filtered from 999,999 in SecLists `xato-net-10-million-passwords-1000000.txt` at commit `c205c36a445bff37f8e58a9ec829105cd4975c58` (MIT) to the 12-to-64-character range the policy can accept, embedded in `MusicRoom.Infrastructure` and loaded once into a case-insensitive set behind `IBreachedPasswordSet`. Both the submitted password and its separator-stripped form are compared, because no entry in the set contains a space while 14,700 of them are a run of words. **Context:** the project's own words plus the address local part and the display name, lowercased with separators removed, refused by containment at four characters or more. Register screens both in its validator; reset reads the account's words **before** its write, so a refusal leaves the token spendable; change reads them from the row it already loads. Login is not screened

Element	Design
Session	15-minute **RS256** JWT access token carrying `sub`, `sid`, `iat`, `exp` and `jti` — `sid` names the refresh-token family, which is what lets logout end one session and leave the others (SPEC-010) — plus an opaque, hashed, rotating refresh token. Validation allows **30 seconds** of clock skew, so a token is accepted for at most 15 minutes 30 seconds. The refresh family's lifetime is **30 days, absolute** — every rotation inherits the family's original expiry, so a stolen family cannot be kept alive by rotating it (owner decision, 2026-09-27, SPEC-005). An **ID token** joins the access token from SPEC-044: RS256, carrying `iss`, `sub`, `aud`, `exp`, `iat`, `auth_time` and `nonce`, five minutes, verifiable against the published JWKS. A session also carries the **scopes granted when it was opened** — a nullable column on its refresh-token family, inherited by every rotation, which is what `GET /oauth2/userinfo` filters on; null means a session from the credential endpoints, where nobody asked for a scope (SPEC-045). It is **not** a claim in the access token, deliberately: that token's claim list is the one above, for the reason stated there. **The access token is RS256 too since SPEC-050**, signed with the same pair and carrying the same `kid`, so one key verifies everything this API issues and `JWT_SIGNING_KEY` no longer exists. No HMAC is accepted, not even during the deploy: an overlap would keep a shared secret that still mints valid tokens, and the cost of refusing is one `401` per active client inside one 15-minute window — refresh tokens are opaque and database-backed, so a client rotates and carries on – [[ADR-0031-musicroom-as-openid-provider]]
Theft detection	reuse of a rotated refresh token revokes the whole family
Verification / reset	single-use, hashed, short-TTL credentials; identical responses whether or not the account exists. **Since SPEC-052** a verification record carries two: a 256-bit link token and a six-character code, both hashed, both spent together, both living 15 minutes, with five attempts against the code before the row is spent. **Since SPEC-055** the reset carries two as well, on the same terms, except that its window stays one hour and the code path answers `400` for an expired row rather than `410`. **Delivery since SPEC-024:** the plaintext token leaves the process only inside the message the sender composes — the row keeps its SHA-256 digest, the provider's API key lives in the typed client, and neither the token nor the key reaches a log event on the hosted path. The Development sender still logs both, which is why it may run nowhere else (T-26, §10.1)

Element	Design
Password change	`POST /api/v1/me/password`, SPEC-028: the current password is the proof ASVS V7.5.1 asks for before a sensitive attribute changes, the new hash and the revocation of every **other** refresh family are one transaction, and the caller's own session survives because it just proved it holds the new credential. A wrong current password is `422`, not `401` — the token is valid, the precondition is not
Social	**OpenID Connect**; one validation core driven by each provider's discovery document, acquisition branching only: Google authorization code + PKCE `S256`, Facebook Limited Login. The ID token is validated server-side including `aud` and `nonce`; one provider identity maps to at most one user – [[ADR-0008-oidc-social-login]]
Provider role	the API is itself an **OpenID Provider**: discovery document, JWKS, authorization endpoint with PKCE `S256`, token endpoint, ID token, UserInfo, revocation. No resource-owner password-credentials grant – [[ADR-0031-musicroom-as-openid-provider]]. **Built as of 2026-10-11**, in three units: SPEC-043 the key, the document and the JWKS; SPEC-044 authorize and token; SPEC-045 UserInfo and revocation. The authorization endpoint **takes a bearer token**, which makes this a first-party provider — ADR-0031's amendment of 2026-10-11

13.2 Authorisation — two independent gates

Both are mandatory and both server-side:

1. **Ownership** — the caller owns the resource or is an authorised participant (R1 V.6).
2. **Visibility and licence** — resolved from `Visibility` and `LicenceMode`, evaluated on every request and every hub join.

Where each gate lives today, which differs from the "one pipeline behaviour" this section originally assumed:

Gate	Mechanism	State
Ownership, own resources	the account comes from the validated token and no route accepts a caller id, so there is nothing to compare — `/me` routes, the session endpoints and the friends graph all work this way	**implemented**
Ownership, somebody else's resource	a resource that is not the caller's answers `404` from the statement that failed to match it, not from a prior check — one `UPDATE` or `DELETE` keyed on the caller	**implemented** for sessions, friend requests and friendships

Gate	Mechanism	State
Per-field visibility	resolved once per read into self / friend / stranger, then applied **inside the SQL projection**: a field the reader may not see is never read, so it cannot leak through a serialiser or a log (OWASP API3) — SPEC-013	**implemented**
Licence, including location and time	[[ADR-0014-location-time-licence]]	designed; arrives with the Track Vote slice
Hub join	the same two gates on every join	designed; arrives with the first hub

A pipeline behaviour is still the right home for the *licence* gate, because it will apply to many commands uniformly. It was the wrong home for per-field visibility, which is a projection rather than a yes-or-no answer.

Row Level Security as deny-by-default defence in depth is **designed and not implemented**: no migration creates a policy. Recorded as issue O19 in §17 — it is defence in depth, so its absence weakens nothing that the gates above already decide, and claiming it while it does not exist would be worse than owing it.

13.3 Abuse resistance

Rate limits keyed on account **and** IP for login, registration, reset, verification resend and vote casting. Progressive backoff on repeated login failure. Generic error text to prevent account enumeration. `429` with `Retry-After`.

The numbers, approved by the repository owner on 2026-09-27 (SPEC-006). Implemented values, not intent: the per-IP half is the built-in middleware, one policy per endpoint; the per-account half is `ICredentialAttemptLimiter`, enforced inside the handler because the address a request names is known only after model binding. Both halves hold state **in process**, which is correct for the single instance [[ADR-0015-hosting-and-deployment]] deploys and would not be shared across several – O8.

Endpoint	Per IP	Per named account
`POST /auth/sessions`	20 / 5 min	5 failures / 5 min, then a penalty of 1 window doubling to a ceiling of 8
`POST /auth/tokens`	60 / 5 min	– the request names no account, only a token
`POST /auth/registrations`	10 / hour	the same per-account counter, charged on **every** attempt
`POST /auth/verification-requests`	10 / hour	the same, charged on every attempt – each one writes a row and spends mail quota (T-19)
`POST /auth/email-verifications`	30 / 5 min	– the request names no account
`POST /auth/password-reset-requests`	10 / hour	the same per-account counter, charged on **every** attempt – SPEC-007
`POST /auth/password-resets`	30 / 5 min	– the request names no account, only a token

Endpoint	Per IP	Per named account
`POST /me/password`	10 / 5 min	the per-account counter, charged on a **wrong** current password and cleared by a correct one — SPEC-028

The realtime surface carries its own three controls since SPEC-026, which implemented the numbers [[ADR-0030-hub-rate-limits-and-origin]] left open until 2026-10-04:

Control	Value	Enforced
Concurrent connections per account	**12**	refused at the handshake (`429` + `Retry-After`) and again at connect. Three devices at three hubs, plus a whole device of reconnect headroom — measured, SPEC-033
Concurrent connections per address	**60**	the same, which is the venue case: twenty devices on one network
Hub invocations per connection	60 a minute	`HubException`, inside the audit filter, recorded `refused`. A realistic client reaches **24** — SPEC-033
Handshake `Origin`	`CORS_ALLOWED_ORIGINS`	`403`; **no** `Origin` passes, which is the native client

The ceiling counts connections, and a device is three of them. `PlaylistHub`, `EventHub` and `ControlHub` are separate connections at separate paths, which SPEC-033's measurement established: one account opened all three and all three were accepted. The first values — 5 per account, 20 per address — were chosen against a count of *devices*, so 5 bought one device and a spare and 20 bought six devices on a shared network. ADR-0030 was amended on 2026-10-06 to 12 and 60, and `docs/load/REPORT.md` §5.6 carries the run behind both: 12 connections held and the 13th refused, against 5 and the 6th before. The invocation limit was left at 60, which a realistic client meets at 24.

Twice for the ceiling, because the two checks answer different problems: a refusal thrown in `OnConnectedAsync` does not reliably reach a client over long polling, and a middleware check reads a count it does not own, so two simultaneous handshakes can both pass it. The middleware gives the client a refusal it can see; the filter holds the authoritative count. Counts are **per process**, as every other limiter here is — O8.

`CORS_ALLOWED_ORIGINS` now governs REST as well. It had been in `.env.example` since S0 and read by nothing, so the handshake check the ADR specifies had no list to compare against; one list rather than two, because the ADR rejected a second allowlist on the grounds that two drift.

Login charges only a *failed* attempt and clears the counter on success. Registration and resend charge every attempt, because the cost is in the attempt rather than in its outcome. The defaults are compiled in and overridable from configuration; a non-positive override is ignored, so configuration cannot switch a limit off.

The vote and venue-code routes carry their own policies, keyed on the **validated account** rather than the address: 60 votes a minute and 10 codes in 5 minutes (SPEC-018). An event is a room full of people on one network, so an address key would make them share one allowance — which is exactly what happened until SPEC-023's load measurement found it. The limiter middleware ran before authentication, so the partition key saw no claims and silently took its address fallback. [[ADR-0041-limiter-after-authentication]] fixed the order: authentication, then the limiter, then authorisation, then the client-context middleware. The limiter still runs before any handler and any database work, so a flood is still refused cheaply — the load report

measures a refusal at about one millisecond. `EventVoteRateLimitingTests` is the regression: two accounts on one address, and the second must still be able to vote.

13.4 Audit log — mandatory, and distinct from application logging

	Audit log	Application log
Purpose	satisfy R1 V.6 — record what a user did	diagnose the system
Store	database table, queryable	structured log sink
Retention	kept; it is evidence	rotated

Audit record: `userId`, `action`, `targetType`, `targetId`, `platform`, `device`, `appVersion`, `occurredAtUtc`, `correlationId`, outcome. See §10.7.

The log inventory — what is written where, which ASVS V16.1.1 asks for and SPEC-027 wrote down:

Layer	Sink	What it carries	What it must never carry
Audit	`audit_records` in PostgreSQL	one row per user action: the ten fields above, including the outcome vocabulary below	a password, a token, a token digest, or a geofence coordinate
Application	container **stdout**, Serilog CLEF (`CompactJsonFormatter`), `@t` in UTC	request lines from `UseSerilogRequestLogging`; start-up decisions; limiter trips with the policy and the wait; unhandled exceptions with their stack; the mail sender's recipient and provider message id; the catalog's call name and its quota cost in units	anything on the never-logged list in [[application-security]] — credentials, tokens, API keys, provider response bodies, and the account behind a rate-limit trip
Traces	OTLP exporter, **off by default** — [[ADR-0022-observability-stack]], issue O16	spans for the request, EF Core and outbound HTTP	the same list
Database	PostgreSQL's own log inside the container	engine errors, such as the `53300` connection exhaustion SPEC-023's run surfaced	not ours to control; it is why `Include Error Detail` stays off, so a constraint violation names no value

Two of the chapter's requirements are **not met** and are recorded as T-33: nothing restricts who may read these logs or makes them immutable (V16.4.2), and nothing ships them to a separate system for alerting (V16.4.3). The exporter above is the mechanism that would, and it is switched off.

The `outcome` vocabulary, implemented values (SPEC-009):

Value	Means
`completed`	the action ran to completion. Also what a handler that says nothing produces
`refused`	wrong credential, or an unknown or already-spent token
`throttled`	the account is out of attempts — §13.3
`expired`	the credential existed and its window had passed
`reusedetected`	an already-rotated refresh token was presented, so the family was revoked — §13.1. The loudest row in the table: a refresh token leaked

`userId` is set where the handler knows the account — a successful sign-in, a rotation, a completed reset — and left null where naming it would say more than the caller proved: a failed login must not record which address exists, and a duplicate registration stays `completed` for the same reason.

Both transports are covered. REST commands go through the pipeline behaviour; SignalR `JoinEvent`, `LeaveEvent`, `JoinPlaylist` and `LeavePlaylist` go through a hub filter writing the same record – [[ADR-0019-hub-action-auditing]]. Client context is captured at the handshake, since a hub invocation carries no HTTP headers, and a handshake without it is rejected. Connect and disconnect are deliberately not audited: mobile reconnect noise would bury deliberate user actions.

13.5 Never logged, never stored, never shipped to a client

Passwords, tokens, refresh tokens, API keys, `Authorization` headers, database connection strings, Supabase keys, OAuth client secrets. A mobile bundle can be unpacked, so it holds none of these.

13.5b Where configuration comes from

Every secret and every setting is read through **one abstraction**, `ISecretProvider`, declared in `Application` and implemented in `Infrastructure` — [[ADR-0029-containerised-dev-and-secret-vault]], SPEC-030. Nothing above `Infrastructure/Configuration` names a configuration source, and `EnvironmentConfiguration` in the composition root is the only caller.

Provider	When	State
`EnvSecretProvider`	the default: the process environment, which `docker compose` fills from `.env`	**built**
`InfisicalSecretProvider`	enabled by configuration, reading a self-hosted Infisical in the same stack	the rest of S1b — issue **O18**

Three rules the abstraction decides once, rather than at the thirteen call sites that used to:

- **A required value that is absent stops start-up**, with a message naming the variable. `DATABASE_URL`, the JWT settings and `MAIL_PROVIDER` are the three guards this preserves.
- **Blank is absent.** A host that writes `MAIL_PROVIDER=` has configured nothing, and a blank number falls back to its approved default instead of failing to convert.
- **A name may appear in a failure message; a value may not** — §13.5, and `secrets-management`'s rule against printing a secret even while debugging. Nothing in the provider logs.

A value that is present and unparseable — `RATE_LIMIT_ACCOUNT_PERMITS=abc` — still stops start-up. The callers clamp what they receive, so a *valid* but useless override is the case that is ignored: a non-positive permit cannot switch a limit off.

13.6 Residual risk

R1 V.6 requires the remaining threats to be identified and their protections described. The register is [[threat-register]]: 18 threats with likelihood, impact, planned mitigation, residual risk, accepted-by and verification, plus an explicit out-of-scope list.

Every unfixed finding from [[owasp-top10-checklist]] or [[owasp-api-top10-checklist]] gets a row there before the pull request merges. An accepted risk that is written down is a defensible engineering decision; an unwritten one is an oversight.

---

14. Deployment and resource viewpoint

14.1 Topology

Element	Where	Note
API	**live on Render**, Frankfurt, free plan, the same image `docker compose` builds – [[ADR-0046-live-deployment-render]]; local Docker on a declared machine for development and measurement – [[ADR-0015-hosting-and-deployment]]	multi-stage image, pinned base, non-root user. The build context is the **repository root**, identical in compose, CI and Render — SPEC-051
Database	S0-S3 a **local PostgreSQL container**; S4 onward Supabase Free tier, Frankfurt – [[ADR-0026-environment-bring-up]]	same engine, same migrations, different connection string
Client	user devices	React Native, built by the second team member after the backend slices land
CI	GitHub Actions, separate `dev` and `prod` configurations, plus an independent client job	builds and tests; **does not deploy**
Merging	`main` requires the `gate` check and `enforce_admins`; **not** that a branch be up to date, since merge queue is unavailable on this plan – [[ADR-0040-pull-requests-need-not-be-up-to-date]]. Auto-merge lands a pull request when `gate` is green	`backend-prod` runs on every push to `main`, so a semantic conflict two green pull requests create together is red within minutes
Observability	Serilog to stdout; OpenTelemetry exporter configuration-driven, **off by default** – [[ADR-0022-observability-stack]]	switched on for a load run

Element	Where	Note
Secrets	`EnvSecretProvider` for the everyday loop; `InfisicalSecretProvider` behind the same abstraction, vault self-hosted in the compose stack under a `vault` profile and **beside the API on Render in production** – [[ADR-0029-containerised-dev-and-secret-vault]], [[ADR-0046-live-deployment-render]]	**required scope.** The abstraction lands early, the vault at S1b, and the delivered system runs on it
Vault store	a **second Supabase project**, separate from the application's – [[ADR-0046-live-deployment-render]]	Render's free PostgreSQL is deleted 30 days after creation, and the database that would vanish holds every secret

There is a live deployment since 2026-10-11 — https://music-room-api-regw.onrender.com, on Render, Frankfurt, free plan, with the vault beside the API and both databases on Supabase ([[ADR-0046-live-deployment-render]], `docs/deploy/RENDER.md`). Vercel was never a candidate: serverless, no long-lived process, no persistent WebSocket.

The free tier sleeps after fifteen minutes and takes about a minute to wake, so a demo warms the stack first — the vault before the API, because the API reads its secrets at start-up and refuses to start without them. **The live deployment is not where the capacity figure comes from**: subject V.7 is answered by the declared machine in `docs/load/REPORT.md`, and a 512 MB shared-CPU container that sleeps is a URL rather than a measurement.

A deployment behind a TLS-terminating proxy must forward its headers, and the live one does (`ASPNETCORE_FORWARDEDHEADERS_ENABLED`). Without it the host reads every request as `http`: the discovery document and the ID token advertise an issuer that does not match the origin they were served from, which a conforming client refuses, and — the quieter half — `RemoteIpAddress` becomes the proxy's, so §13.3's per-IP credential limit partitions the whole internet into one bucket. Both were found by driving the live deployment; neither is visible from inside a test host, because a test host is not behind a proxy.

The database being off-premise must be stated in the load report: network latency to Supabase is part of the measured numbers, not an excuse for them.

14.2 Reproducibility

Subject R1 IV.1 requires dependencies to install automatically from a fresh clone. The path:

```
clone  ->  cp .env.example .env  ->  fill values  ->  make up
```

`Makefile` and `docker-compose.yml` exist, and the path above is verified end to end: the API container serves `/health` and `/health/ready` against the compose PostgreSQL. Built in slice S0 of [[delivery-plan]]; rules in [[ci-cd-containerization]]. Issue O10 is closed. CI builds and tests but does not deploy – [[ADR-0015-hosting-and-deployment]].

14.2.1 Checking a deployment

A deployment can be wrong in ways no test can be: a variable nobody set, a proxy that hides the scheme, a database that is not migrated. `make smoke` — SPEC-053 — checks exactly those, against any base URL, in a few seconds: liveness, every member of the readiness body, the discovery document's issuer **including its scheme**, the published key set, and, with credentials for one already-verified account, a signed-in

read and one catalog search. It creates nothing and sends no mail; the suite proves the rules, and this proves the configuration they run under. `make smoke-live` aims it at the hosted service. The runbook is `docs/deploy/RENDER.md`.

Two live defects are the reason it exists. The issuer said `http://` behind a TLS-terminating proxy until `ASPNETCORE_FORWARDEDHEADERS_ENABLED` was set, which also silently collapsed the per-IP credential limit into one bucket. And `YOUTUBE_API_KEY` was never set on the service, so every `GET /tracks` answered `503` with nothing in the log to say why. `/health/ready` now reports `catalog` — **configuration only, never a call**, because a `search.list` costs 100 of 10,000 daily quota units — and `Degraded` there never makes readiness fail, because only search touches the provider. The provider's log now distinguishes four causes of a catalog `503`: no key, a key the provider rejects, the daily quota, and the provider's own trouble. **The same treatment reached social sign-in on 2026-11-03** — SPEC-056: the four `GOOGLE_*` variables were never set on the live service, so every `POST /auth/sessions/social` answered `422 PROVIDER.UNKNOWN`, and the way that came to light was the front-end team reporting it as a defect in the API. A refusal to a client is the wrong place for a deployment to learn about its own configuration, so `/health/ready` now carries a `social` member — the provider rows by name, never a credential, never a call to the provider — and a start-up line names all four variables when there is no row. The response is identical in all four; which one applied is the deployment's business, not the caller's.

14.3 Configuration

Development is **fully containerised**: `docker compose up` brings up the stack and the host needs Docker and nothing else – [[ADR-0029-containerised-dev-and-secret-vault]]. No documented step requires a locally installed SDK, database client or CLI; `dotnet-ef` is pinned through `.config/dotnet-tools.json`.

Secrets **go into** the vault by script — `make vault-seed FILE=.env.prod`, SPEC-054. It writes an allowlist of names over Infisical's API, idempotently, printing names and outcomes and never a value, and it refuses to run under shell tracing because a trace would echo every one. Typing ten values into a web form was the step in the live migration most likely to go wrong, and a mistyped `OIDC_SIGNING_KEY` — a PEM block carried as one line of `\n` escapes — costs an afternoon rather than a deploy. The five `INFISICAL_*` values are not among the names it writes: they are what opens the vault.

Secrets are read through a provider abstraction, so the vault replaces the environment without touching a call site. The vault is **required scope** – [[ADR-0029-containerised-dev-and-secret-vault]], amended 2026-09-27 – and it **landed at S1b**: SPEC-030 the abstraction, SPEC-042 the vault in this compose stack, SPEC-047 the provider that reads from it. It **shrinks** `.env` to the vault's own bootstrap credentials; it does not remove it, and subject V.8 still applies to what remains.

`SECRET_PROVIDER` selects: `env` (the default, and what CI uses) or `infisical`. Three properties of that switch are load-bearing, and each is asserted by a test rather than described here only. **The vault outranks `.env`, and `.env` is the floor beneath it** – forced rather than chosen, because `INFISICAL_CLIENT_ID` and `INFISICAL_CLIENT_SECRET` are what the application logs in with and so cannot live in the vault they open. **A vault that cannot be reached stops the host**, because a fallback would silently return the system to reading API keys out of a file and nothing in a log would say the posture had changed. And **secrets are read once, at start-up**, so a read costs no network, a vault outage is a boot failure rather than a request failure, and rotating a secret takes a restart – the honest cost, written beside the variable in `.env.example`.

All configuration from environment variables, named after the role rather than the vendor (`DATABASE_URL`, not a provider name) – [[ADR-0026-environment-bring-up]]. External credentials are created just in time: mail before S1, YouTube and the Supabase project before S4, Google and Facebook before S8. Every key is listed in `.env.example` with a placeholder. `.env` is git-ignored. R1 V.8 makes a publicly stored credential an automatic project failure.

14.4 Capacity

Tool: **Gatling**, simulations committed under `tests/load/` – [[ADR-0018-load-testing-tool]]. It is named in the subject and supports WebSocket natively, so one methodology covers both REST throughput and

SignalR fan-out. It runs as `gatling-maven-plugin` inside `maven:3-eclipse-temurin-21` and not from a Gatling image, because no official Gatling image resolves; SPEC-023 records the deviation with its probe evidence.

Measured on 2026-10-03 — `docs/load/REPORT.md`. Five scenarios, five steps each from 10 to 200 concurrent, against one API container and one PostgreSQL container on an Apple M5 Pro whose Docker VM had 15 CPUs and 7.75 GiB. The stated figure is **100 concurrent users performing contended writes, and 200 reading or listening, on that machine**. REST throughput and SignalR connection fan-out are measured separately — they are different limits. See [[load-testing]].

The bottleneck is **PostgreSQL's connection count, not CPU**: at 200 concurrent collaborators a quarter of the reads answered `500` with `53300: sorry, too many clients already` while 62% of the VM's CPU was still idle. Npgsql's default maximum pool size equals the server's default `max_connections`, so the pool may ask for more connections than the server will give. The pair of numbers that fixes it is a deployment decision and is issue O22.

The measurement also found a defect in shipped code: the rate limiter ran before authentication, so SPEC-018's per-account vote policy partitioned on the remote address and a whole venue shared one allowance. [[ADR-0041-limiter-after-authentication]] fixes the order, §13.3 records it, and the re-measurement is in the report.

The figure above is for this machine and this topology only. The API is on-premise and the database is off-premise once S4 lands: latency to Frankfurt will be inside every number measured then, and Supabase's pooler changes the connection ceiling entirely, so that run is a new measurement rather than a correction of this one.

---

15. Design rationale

Rationale is not restated here. Each decision has its own record with the alternatives that were rejected and why — evaluators ask "why not X", and the answer must be on file.

ADR	Decision	Status
[[ADR-0001-backend-dotnet-narchitecture]]	.NET 10, Clean Architecture + CQRS	Accepted
[[ADR-0002-mobile-react-native]]	React Native client, deferred	Accepted
[[ADR-0003-music-provider-youtube]]	YouTube as the music provider	Accepted
[[ADR-0004-realtime-signalr]]	SignalR over WebSocket	Accepted
[[ADR-0005-api-style-rest-json]]	REST + JSON + OpenAPI	Accepted
[[ADR-0006-datastore]]	Supabase managed PostgreSQL	Accepted
[[ADR-0007-password-hashing-bcrypt]]	BCrypt for password hashing	Accepted
[[ADR-0008-oidc-social-login]]	OpenID Connect for Google and Facebook	Accepted
[[ADR-0009-code-quality-gates]]	OWASP conformance and enforced code-size limits	Accepted
[[ADR-0010-service-scope]]	Ship all three services	Accepted
[[ADR-0011-solution-layout]]	Layout mirrors nArchitecture	Accepted
[[ADR-0012-test-stack]]	xUnit, NSubstitute, Testcontainers	Accepted
[[ADR-0013-mail-provider]]	Hosted transactional mail provider	Accepted
[[ADR-0014-location-time-licence]]	Geofence plus optional event code	Accepted

ADR	Decision	Status
[[ADR-0015-hosting-and-deployment]]	Local Docker, CI without deployment	Accepted
[[ADR-0016-supabase-plan-region]]	Supabase Free tier, Frankfurt	Accepted
[[ADR-0017-playback-session]]	PlaybackSession aggregate, client-driven advance	Accepted
[[ADR-0018-load-testing-tool]]	Gatling for load testing	Accepted
[[ADR-0019-hub-action-auditing]]	Audit hub actions with the same record	Accepted
[[ADR-0020-monorepo-layout]]	Backend and client in one repository	Accepted
[[ADR-0021-backend-dependency-set]]	MediatR, FluentValidation, built-in rate limiting and OpenAPI	Accepted
[[ADR-0022-observability-stack]]	Serilog with OpenTelemetry	Accepted
[[ADR-0023-catalog-cache]]	Two-level catalog cache	Accepted
[[ADR-0024-expiry-and-cleanup]]	Lazy expiry checks, one cleanup service	Accepted
[[ADR-0025-agent-git-protocol]]	Git protocol for unattended agent runs	Accepted
[[ADR-0026-environment-bring-up]]	Environment bring-up sequence	Accepted
[[ADR-0027-agent-push-and-pr]]	Agents push a branch and open a draft pull request	Accepted
[[ADR-0028-domain-owns-its-base-types]]	Domain owns its base types; Application owns its abstractions	Accepted
[[ADR-0029-containerised-dev-and-secret-vault]]	Containerised development, self-hosted secret vault	Accepted
[[ADR-0036-access-token-mechanics]]	JwtBearer, a `sid` claim, 30 seconds of clock skew	Accepted
[[ADR-0037-credential-rate-limiting]]	Per-IP middleware plus a per-account in-process counter	Accepted
[[ADR-0032-refresh-token-lifetime]]	30-day absolute refresh-token families	Accepted
[[ADR-0033-audit-write-posture]]	The audit record commits with its action, and fails closed	Accepted
[[ADR-0034-profile-storage-and-visibility]]	Typed profile columns with a tier each	Accepted
[[ADR-0038-realtime-broadcast-posture]]	A handler queues a broadcast; a behaviour sends it after the commit	Accepted
[[ADR-0039-queue-broadcast-payloads]]	An `EventHub` payload carries the authoritative value, not a row version	Accepted

An accepted ADR is never edited. It is superseded by a new one.

---

16. Requirements traceability

A row marked `- continued` carries the rest of the row above it. The split is typographic rather than semantic — a single cell taller than a page cannot be typeset in a long table, and this document is distributed as a PDF.

Every mandatory requirement of R1, mapped to the design element that satisfies it. This is the table to walk through at defence.

Subject	Requirement	Design element	Status
Subject	Requirement	Design element	Status
V.1	email+password or social registration	§10.1, §10.2, §13.1	**partly implemented** – `POST /api/v1/auth/registrations` persists an unverified `User` with a BCrypt hash at work factor >= 12, `ux_users_email` arbitrates the race, and both a fresh and an already-registered address answer `202` so the endpoint is not an enumeration oracle. Social registration is designed only. A verified account can now sign in, and **the API can tell who is calling**: `AddJwtBearer` validates the signature, issuer, audience and lifetime of the token this service signed, and `DELETE /auth/sessions/current`, `GET /auth/sessions` and `DELETE /auth/sessions/{sessionId}` work — logout revokes the refresh-token family named by `sid`, so a kept copy of the refresh token is worthless afterwards. `POST /api/v1/auth/sessions` verifies the stored hash and answers a 15-minute JWT plus an opaque refresh token, and every failure – unknown address, wrong password, unverified account – answers the same bodiless `401` for the same amount of hashing work. Rate limiting is **in place** on this endpoint and on every other credential endpoint — per-IP policy plus a per-account counter, §13.3 — so T-25 is closed. **A chosen password is now screened** (SPEC-035): every path that chooses one — register, reset and the

Subject	Requirement	Design element	Status
V.1	— continued	§10.1, §10.2, §13.1	**Update 2026-10-10, SPEC-041:** social registration exists. `POST /api/v1/auth/sessions/social` exchanges an authorization code at the provider's token endpoint with the client secret — this API exchanges it, never the client — validates the ID token against the provider's JWKS, and creates an account the first time an identity signs in. That account has no password, so `users.password_hash` is nullable; password sign-in for it answers the same generic `401`, and `POST /me/password` answers `422`. An address the provider has not verified is refused rather than matched, which is ADR-0008's account-takeover path.

Subject	Requirement	Design element	Status
V.1	— continued	§10.1, §10.2, §13.1	**Update 2026-11-03, SPEC-057:** so is a **verified** address that already belongs to an account this identity is not linked to — the same refusal, now answered 422SOCIAL.ADDRESS_HAS_ACCOU of a bodiless401, because the provider's own verification means the caller has proved they control the address and learns nothing a password-reset request would not tell them. The message names the way through - sign in with the password, thenPOST /auth/social-links- which is what the front-end team had no way to show when the refusal was indistinguishable from a bad code. Facebook is the second acquisition against the same core and is not built | | V.1 | link a social account | §10.2, Identity context | **implemented** **SPEC-041, 2026-10-10.**GET,POSTandDELETE /api/v1/auth/social-links. Linking uses the same acquisition as signing in, deliberately: an identity the caller cannot prove they hold would let anyone attach their provider account to somebody else's. The identity is the pair(provider, sub), unique in the database, and an address is a label rather than a key

Subject	Requirement	Design element	Status

"Designed" means the decision and the mechanism are recorded, and nothing more. "Partly implemented" and "done" name code that exists on a branch and passes `make agent-check`; the row says which part, because a requirement is rarely finished in one slice.

---

17. Open design issues

Closed rows stay visible: a decision log that hides what it used to not know is less useful at defence, not more.

Open

#	Issue	Blocks	Note
O8	SignalR scale-out backplane (Redis)	multi-instance deployment	**deliberately open.** Not needed for a single-instance defence
O12	fluent assertion library	test readability	add only if xUnit's built-ins prove insufficient
O13	silent-controller recovery	a stalled playback session	[[ADR-0017-playback-session]] accepted no watchdog; takeover is the manual path. Revisit only if it bites in testing
O14	music preferences have no consumer	nothing	V.1 requires storing and updating them, which is met. Note for defence: storage-only today
O19	RS256 key pair: generation, storage, JWKS exposure and rotation with an overlap window	the OpenID Provider surface at S8	[[ADR-0031-musicroom-as-openid-provider]]. `EnvSecretProvider` holds the private key until the vault lands at S1b, after which it belongs there. Rotation needs an overlap so tokens signed by the outgoing key keep verifying

#	Issue	Blocks	Note
O20	PostgreSQL Row Level Security policies are not written	nothing — it is defence in depth	§13.2 claimed RLS was enabled; no migration creates a policy. Opened by the SPEC-013 SDD sync on 2026-09-29. Deny-by-default policies on the per-account tables would mean a query that forgot its `WHERE` returns nothing rather than everything. Cost: every read path needs a session variable carrying the caller, which the current single-connection-pool design does not set
O22	Npgsql pool size and PostgreSQL `max_connections` are not set against each other	the V.7 figure above 100 concurrent writers	opened by SPEC-023's measurement on 2026-10-03. Both default to 100 and the server reserves superuser slots, so the pool may ask for connections the server will not give and the request fails with `53300` instead of queueing. Capping the pool below `max_connections` fixes the failure mode; raising `max_connections` moves the ceiling. **Not fixed here on purpose:** S4 moves `DATABASE_URL` to Supabase, whose pooler changes the arithmetic, and [[load-testing]] requires the change and its re-measurement in one unit

#	Issue	Blocks	Note
O23	No backup of the development database	**closed**	`pg_dump` of the compose Postgres into a directory, with a retention count and a restore path. Needs an ADR before a spec, because the spec gate trips twice: question 1 (no accepted decision covers it) and question 5 (`pg_dump` reads the `.env` credential, and a dump carries real user rows — hashes and addresses). Three things the ADR has to settle: where the directory lives and that it is git-ignored **in the same commit** (V.8 makes a committed credential an automatic failure); that the dump runs as `docker compose exec db pg_dump` rather than a host binary, since `psql` is absent from the toolchain [[MEMORY]] records and [[ADR-0026-environment-bring-up]] keeps the host empty; and how a restore is proved, because a backup nobody restored is a file **Closed 2026-10-09 by SPEC-040**, under [[ADR-0044-development-database-backup]]. `make backup` dumps through `docker compose exec -T db pg_dump`, never a host binary and never with the credential on the command line; `backups/` was git-ignored in the commit that created the target; seven dumps are kept and the same command prunes the rest. The restore was run rather than assumed: a marker row inserted, dumped and read back out of a restored scratch database, and the guard seen refusing a `DATABASE_URL` that names Supabase

#	Issue	Blocks	Note
O24	The composition root is at the file-size cap, so every unit that touches the pipeline pays an extraction first	the next unit that adds a line to `Program.cs`	opened by SPEC-032 on 2026-10-06. `Program.cs` has been at or near S104's 300-line cap for three units running — SPEC-024, SPEC-025 and SPEC-026 each had to extract a class to make room, and SPEC-032 was **blocked** by it until a file was added to its Files table: four lines took the file to 304 and `Main`'s coupling to 42 against CA1506's 41. The options, none of them this spec's to choose: extract the remaining helper classes — `ApiServices`, `RealtimeServices`, `Observability`, `EnvironmentConfiguration`, `InfrastructureSetup`, `CallerServices` — into their own files, which is a refactor with no behaviour change and a large diff; or decide that **300 is the wrong number for a composition root** and raise S104 for that file or that folder, which is a standards change and therefore an ADR. Left open deliberately: a cap raised to fit a change is what `.editorconfig` exists to prevent

#	Issue	Blocks	Note
O25	The realtime ceiling counts connections and means devices	nothing today; the next hub	opened by SPEC-033 on 2026-10-06. A client holds one connection per hub, so the per-account ceiling has to be multiplied by the hub count every time a hub is added: three hubs made 12 mean three devices, and a fourth would make it two and a quarter. The handshake already carries `X-Client-Device` (SPEC-016), so the registry could count **devices** and the number would then mean what its rationale says — one ceiling that survives a new hub. Cost: the registry keys on a value the client supplies, so two devices claiming one name would share a slot, which is a trade a design decision has to weigh rather than a number. Left open deliberately: this is an ADR, not a spec

#	Issue	Blocks	Note
O26	The `422` validation body is a published contract with no test behind it	**closed**	opened by SPEC-034 on 2026-10-06, with the numbers rather than a feeling: **37 assertions** across the suites check `HttpStatusCode.UnprocessableEntity` and **zero** checked the body's `errors` keys, while the committed `docs/api/openapi.json` publishes `HttpValidationProblemDetails` on **22 operations**. `CredentialEndpointResults.ToErrors` shapes that body for **21 call sites in 14 endpoint files**, and it groups failures by the property name FluentValidation reported against — so renaming a validated property changes a response shape clients read and **nothing fails**. The OpenAPI drift guard cannot help: the schema describes the envelope, not the keys inside `errors`. SPEC-034 added the first three assertions of their kind, on the password paths it refactored, and deliberately did not fix the rest: a unit that closes a repository-wide gap while refactoring a validator is two changes in one pull request. **Closed 2026-10-06 by SPEC-036.** All **20** routes that publish an `errors` body now assert their keys: nine on tests that already existed, eight on new tests, three already done by SPEC-034 and SPEC-035. The three four-line body readers those units left became one shared `ValidationProblems`, so a fourteenth suite asserting a key costs one line rather than five. Handler-produced 422s

#	Issue	Blocks	Note
O27	`UserRepository.cs` is at the file-size cap, so the next account query pays a new file first	the next unit that needs a method on `IUserRepository`	opened by SPEC-035 on 2026-10-06, recorded the way O24 records `Program.cs`. The file is at **295 code lines of S104's 300** after this unit widened two projections; it was at 293 before. The reset path's context read is one more query method — twelve code lines with its projection and its documentation — so it went into `PasswordContextRepository.cs` instead, on the owner's answer of 2026-10-06: a new file, never a raised cap. That answer scales to exactly one more query. The options, none of them a spec's to choose: split the file along its aggregates, which is a refactor with a large diff and fourteen methods every auth suite depends on; or decide that 300 is the wrong number for a repository, which is a standards change and therefore an ADR. **The same wall stands in the test project:** `PasswordResetEndpointTests.cs` is at 293 of 300, which is why SPEC-035's reset screening became a theory case on an existing test rather than a test of its own

#	Issue	Blocks	Note
O28	The `errors` keys of a `422` are three conventions, not one, and one of them is unreadable	**closed**	opened by SPEC-036 on 2026-10-06, from the assertions it added rather than from reading. Three facts. **One:** event create, update and licence publish **camelCase** keys — `endsAtUtc`, `presenceProof`, `geofence`, `geofence.latitude` — because `EventLicenceRules` calls `.WithName(...)`, while every other route publishes the **PascalCase** property name: `Password`, `NewPassword`, `Licence`, `Name`, `TargetUserId`, `GranteeUserId`, `YoutubeVideoId`, `Query`, `PageSize`, `Email`, `TimeToLiveSeconds`. A client reading `errors` has to know which route it asked. **Two:** `MoveTrackCommandValidator` declares its "exactly one neighbour" rule as `RuleFor(command => command)` with no `WithName`, so the published body on `PATCH /playlists/{playlistId}/tracks/{pla` is `{"errors":{"":["Send exactly one of afterTrackId or beforeTrackId."]}}` — an **empty key**, which no client can act on. SPEC-036 covered that route through the self-move rule instead and asserted nothing about the empty one, because renaming a published key is a contract change and not a test's business. **Three:** `docs/api/ENDPOINTS.md` does not list `422` for `POST /auth/password-reset-requests`, which produces one — the route declares `ProducesValidationProblem`

#	Issue	Blocks	Note
O28	— continued	**closed**	The options, none of them a spec's to choose: name every rule explicitly in one casing, which changes keys clients may already read; or publish the convention in `docs/api/ENDPOINTS.md` and keep it. Either is a decision about a published contract, so it is an ADR. **Closed 2026-10-06 by SPEC-037**, implementing [[ADR-0043-validation-error-key-naming]] as amended: one resolver in `ApplicationServiceRegistration` camelCases every member path, `OverridePropertyName` states the key where a resolver cannot reach it, and `ValidationKeyContractTests` asserts every key against the request schema in `docs/api/openapi.json` rather than against a list of strings. 21 expectations over 14 keys moved to the wire names; the empty key on `PATCH /playlists/{playlistId}/tracks/{pla` became `afterTrackId`; `genres.count` became `genres`; `q` and `ttlSeconds` are stated because the property and the field are spelled differently. §Conventions in `docs/api/ENDPOINTS.md` now carries the rule, and that route's missing `422` is listed. **Two things the unit found:** the Application-layer validator tests asserted PascalCase keys and passed because they never applied the global resolver — 21 of 132 fail without the module initializer that now applies it — and two

#	Issue	Blocks	Note
O16	OpenTelemetry must earn its place	credibility of the V.7 bottleneck claim	[[ADR-0022-observability-stack]]: capture one trace during the Gatling run showing EF Core, HttpClient and total request time together

Closed

#	Issue	Closed by
O21	hub call limits and handshake origin decided and not built	**SPEC-026, 2026-10-04.** The ceiling, the per-connection invocation limit and the `Origin` check all exist, with the numbers the owner set in [[ADR-0030-hub-rate-limits-and-origin]] on the same day. `CORS_ALLOWED_ORIGINS` is read by code for the first time, which is the half of this issue the brain-check of 2026-10-02 found. T-12 narrows from "bounded payloads and timeouts" to a stated limit with tests behind it; the per-process residual is O8, unchanged
O11	remote deployment target	**ADR-0046 and SPEC-051, closed 2026-10-11.** Render, Frankfurt, free plan: the API and a self-hosted Infisical as two Docker services, a Key Value cache, and both databases on Supabase. Chosen from the three free tiers the owner named on one fact — it is the only one that can run a second Docker service for free, and the vault is a second Docker service. The costs are in the ADR rather than discovered: the tier sleeps, the vault becomes a boot-order dependency the API retries for ninety seconds and then still refuses, and the capacity figure stays on the declared machine

#	Issue	Closed by
O32	the access token was HS256 while the ID token was RS256	**SPEC-050, closed 2026-10-11.** `JwtAccessTokenIssuer` signs with `ISigningKeyProvider.Current` and the handler accepts every key the JWKS publishes, so a rotation overlap covers access tokens as well. `JWT_SIGNING_KEY` is removed from the code, the test host, `.env.example` and the document build — a secret a project asks for and never uses is configuration nobody can audit. **No HMAC overlap**, deliberately: it would have kept a shared secret that still mints valid tokens plus a knob to remember to turn off, and the cost of refusing is one `401` per active client in one 15-minute window, with refresh tokens untouched
O30	the document build's environment was configured in three places	**SPEC-048, closed 2026-10-11.** Both workflows call `make openapi`, so the Makefile is the one definition of what the generator needs; CI keeps its `--no-restore` through `OPENAPI_BUILD_FLAGS`, which defaults to empty so a local run is unchanged. The trap this closes caught three units in a row — SPEC-005's JWT settings, SPEC-024's mail guard and SPEC-043's signing key — each adding a start-up requirement in one place and failing in another. `grep` over `.github/workflows/` now finds no copy of the generator's environment

#	Issue	Closed by
O18	self-hosted Infisical vault	**SPEC-030, SPEC-042 and SPEC-047, closed 2026-10-11.** `ISecretProvider` with two implementations; a self-hosted Infisical in this compose stack behind a `vault` profile, so `make up` stays two services and a defence walkthrough brings the vault up; and `InfisicalSecretProvider`, selected by `SECRET_PROVIDER`, authenticating with a machine identity. The vault outranks `.env` and `.env` remains the floor — forced, because the vault's own bootstrap credentials cannot live inside it, which ADR-0029 wrote down before any of it was built. A vault that cannot be reached stops the host instead of falling back, because a fallback would silently restore the posture the vault exists to remove. Secrets are read once at start-up, so rotation needs a restart
O17	real mail provider adapter	**SPEC-024, 2026-10-04.** `BrevoMailService` behind `IMailService`, selected by `MAIL_PROVIDER`; the Development logging sender stays for local work and is refused anywhere else. Verification and reset messages now reach a real mailbox, from Brevo's sandbox sender (§10.1). Closes T-26 with it
O1	which services ship	[[ADR-0010-service-scope]] — all three
O9	concrete table and column definitions	the migrations, not a document: `InitialIdentity`, `RefreshTokens`, `PasswordResetTokens` and `AuditRecords` ([[migrations-index]]). `SchemaConventionTests` holds the built model to the naming rules
O15	MediatR licence terms	[[ADR-0021-backend-dependency-set]] records the verification of 2026-09-24: the free Community License covers this project, so MediatR stays and the hand-written dispatcher is not needed

#	Issue	Closed by
O10	`Makefile` and `docker compose`	slice S0 — both exist, `make up` verified end to end against the compose PostgreSQL
O2	solution and folder layout	[[ADR-0011-solution-layout]]
O29	An unhandled exception answers a bare `500` with no `ProblemDetails` body	**closed**
O31	The end-to-end check is not in CI	before the client moves to the code flow
O3	test framework	[[ADR-0012-test-stack]]
O4	API hosting target	[[ADR-0015-hosting-and-deployment]]
O5	Supabase plan tier and region	[[ADR-0016-supabase-plan-region]]
O6	SMTP provider	[[ADR-0013-mail-provider]]
O7	location+time licence enforcement	[[ADR-0014-location-time-licence]]
G1	queue advance / playback progression	[[ADR-0017-playback-session]]
G2	what a control delegate actually controls	[[ADR-0017-playback-session]]
G3	load testing tool	[[ADR-0018-load-testing-tool]]
G4	hub actions missing from the audit log	[[ADR-0019-hub-action-auditing]]

Related

[[HOME]] · [[MEMORY]] · [[system-overview]] · [[domain-model]] · [[realtime-architecture]] · [[auth-and-identity]] · [[decisions-index]] · [[subject-requirements]] · [[technical-guide-index]] · [[user-manual]]